

BPE Implement করা

এখন আমরা **BPE (Byte-Pair Encoding)** অ্যালগরিদমের একটি **implementation** দেখব।

এটা কোনো **optimized version** না, অর্থাৎ বড় corpus-এর ওপর বাস্তবে চালানোর জন্য এটা best version না। এখানে মূল উদ্দেশ্য হলো কোড দেখে অ্যালগরিদমটা ভালোভাবে বোঝা।

১) প্রথমে একটি **corpus** লাগবে

আমরা কয়েকটি sentence নিয়ে একটি ছোট corpus বানাই:

```
corpus = [  
    "This is the Building-LLMs-from-scratch-In-Bangla Course.",  
    "This chapter is about tokenization.",  
    "This section shows several tokenizer algorithms.",  
    "Hopefully, you will be able to understand how they are trained and generate tokens.",  
]
```

এর মানে কী?

এখানে **corpus** হলো কিছু টেক্সটের collection, যেগুলো থেকে BPE শিখবে।

২) এরপর **corpus**-টাকে **pre-tokenize** করতে হবে

যেহেতু আমরা GPT-2 এর মতো একটি BPE tokenizer বানাতে চাই, তাই pre-tokenization-এর জন্য GPT-2 tokenizer ব্যবহার করা হচ্ছে।

```
from transformers import AutoTokenizer
```

```
tokenizer = AutoTokenizer.from_pretrained("gpt2")
```

সহজভাবে:

এখানে GPT-2 tokenizer লোড করা হচ্ছে, যাতে টেক্সটকে শুরুতে word-level বা basic unit-এ ভাঙা যায়।

৩) প্রতিটি **word** কতবার এসেছে, সেটা গণনা করা

```
from collections import defaultdict
```

```
word_freqs = defaultdict(int)
```

```
for text in corpus:
```

```
    words_with_offsets = tokenizer.backend_tokenizer.pre_tokenizer.pre_tokenize_str(text)
```

```
    new_words = [word for word, offset in words_with_offsets]
```

```
    for word in new_words:
```

```
        word_freqs[word] += 1
```

```
print(word_freqs)
```

এখানে কী হচ্ছে?

- **corpus**-এর প্রতিটি sentence নেওয়া হচ্ছে
- GPT-2 tokenizer দিয়ে সেটাকে pre-tokenize করা হচ্ছে
- প্রতিটি word/token কতবার এসেছে, সেটা count করা হচ্ছে

Output:

```
defaultdict(int, {  
    'This': 3,  
    'Gis': 2,  
    'Gthe': 1,  
    ...  
})
```

Ġ চিহ্নটা কী?

GPT-2 tokenizer-এ Ġ সাধারণত **space**-এর চিহ্ন বোঝায়।

যেমন:

- Ġis মানে " is"
- Ġthe মানে " the"

অর্থাৎ token-এর আগে space ছিল।

৪) Base vocabulary তৈরি করা

এখন corpus-এ ব্যবহৃত সব **character** সংগ্রহ করে base vocabulary বানানো হবে।

```
alphabet = []

for word in word_freqs.keys():
    for letter in word:
        if letter not in alphabet:
            alphabet.append(letter)
alphabet.sort()

print(alphabet)
```

Output:

```
[' ', '.', 'C', 'F', 'H', 'T', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'k', 'l', 'm', 'n', 'o', 'p', 'r', 's',
 't', 'u', 'v', 'w', 'y', 'z', 'G']
```

মানে কী?

এখানে corpus-এর সব unique character নিয়ে alphabet বানানো হয়েছে।

৫) Special token vocabulary-তে যোগ করা

GPT-2 model একটি special token ব্যবহার করে:

```
"<|endoftext|>"
```

তাই সেটাকে vocabulary-র শুরুতে যোগ করা হচ্ছে।

```
vocab = ["<|endoftext|>"] + alphabet.copy()
```

মানে:

এখন vocabulary শুরু হবে special token দিয়ে, তারপর alphabet-এর সব character থাকবে।

৬) প্রতিটি word-কে character-এ split করা

BPE training শুরু করতে হলে প্রতিটি word-কে প্রথমে character level-এ ভাঙতে হবে।

```
splits = {word: [c for c in word] for word in word_freqs.keys()}
```

উদাহরণ:

যদি word হয় "This", তাহলে split হবে:

```
['T', 'h', 'i', 's']
```

৭) Pair frequency বের করার function

এখন BPE training-এর গুরুত্বপূর্ণ ধাপ হলো:

কোন দুইটা পাশাপাশি character/token সবচেয়ে বেশি বার এসেছে, সেটা বের করা।

তার জন্য function:

```
def compute_pair_freqs(splits):
    pair_freqs = defaultdict(int)
    for word, freq in word_freqs.items():
        split = splits[word]
        if len(split) == 1:
            continue
        for i in range(len(split) - 1):
            pair = (split[i], split[i + 1])
            pair_freqs[pair] += freq
    return pair_freqs
```

এখানে কী হচ্ছে?

- প্রতিটি word-এর split list নেওয়া হচ্ছে
- পাশাপাশি দুইটি token-এর pair বানানো হচ্ছে
- word যতবার corpus-এ এসেছে, pair-এর frequency ততবার বাড়ছে

৮) কিছু pair frequency দেখা

```
pair_freqs = compute_pair_freqs(splits)

for i, key in enumerate(pair_freqs.keys()):
    print(f"{key}: {pair_freqs[key]}")
    if i >= 5:
        break
```

Output:

```
('T', 'h'): 3
('h', 'i'): 3
('i', 's'): 5
('Ğ', 'i'): 2
('Ğ', 't'): 7
('t', 'h'): 3
```

অর্থ:

যেমন:

- ('T', 'h') pair 3 বার এসেছে
- ('Ğ', 't') pair 7 বার এসেছে

৯) সবচেয়ে **frequent pair** বের করা

```
best_pair = ""
max_freq = None

for pair, freq in pair_freqs.items():
    if max_freq is None or max_freq < freq:
        best_pair = pair
        max_freq = freq

print(best_pair, max_freq)
```

Output:

```
('Ğ', 't') 7
```

মানে:

সবচেয়ে বেশি ব্যবহৃত pair হলো:

('G', 't')

অর্থাৎ " t" জাতীয় pattern সবচেয়ে বেশি এসেছে।

১০) প্রথম merge rule শেখা

তাই প্রথম merge rule হবে:

('G', 't') -> 'Gt'

এটাকে store করা হচ্ছে:

```
merges = {("G", "t"): "Gt"}  
vocab.append("Gt")
```

মানে:

এখন vocabulary-তে নতুন token "Gt" যোগ হলো।

১১) Merge apply করার function

এখন নতুন merge rule-টা সব word-এর split-এ apply করতে হবে।

```
def merge_pair(a, b, splits):  
    for word in word_freqs:  
        split = splits[word]  
        if len(split) == 1:  
            continue  
  
        i = 0  
        while i < len(split) - 1:  
            if split[i] == a and split[i + 1] == b:  
                split = split[:i] + [a + b] + split[i + 2 :]  
            else:  
                i += 1  
        splits[word] = split  
    return splits
```

সহজভাবে:

এই function যা করে:

- প্রতিটি word-এর token list দেখে
- যদি পাশাপাশি **a** আর **b** পাওয়া যায়
- তাহলে সেগুলো merge করে **a+b** বানায়

১২) প্রথম **merge**-এর ফল দেখা

```
splits = merge_pair("Ġ", "t", splits)
print(splits["Ġtrained"])
```

Output:

```
['Ġt', 'r', 'a', 'i', 'n', 'e', 'd']
```

মানে:

আগে "**Ġtrained**" split থাকলে সেটা ছিল:

```
['Ġ', 't', 'r', 'a', 'i', 'n', 'e', 'd']
```

এখন **Ġ** আর **t** merge হয়ে হয়েছে:

```
['Ġt', 'r', 'a', 'i', 'n', 'e', 'd']
```

১৩) **Loop** চালিয়ে বারবার **merge** শেখা

এখন এই process বারবার চলবে যতক্ষণ না vocabulary desired size-এ পৌঁছায়।

```
vocab_size = 50
```

```
while len(vocab) < vocab_size:
    pair_freqs = compute_pair_freqs(splits)
    best_pair = ""
```

```

max_freq = None
for pair, freq in pair_freqs.items():
    if max_freq is None or max_freq < freq:
        best_pair = pair
        max_freq = freq
splits = merge_pair(*best_pair, splits)
merges[best_pair] = best_pair[0] + best_pair[1]
vocab.append(best_pair[0] + best_pair[1])

```

এখানে **loop** কী করছে?

প্রতিবার:

1. pair frequency বের করছে
2. সবচেয়ে frequent pair খুঁজছে
3. pair merge করছে
4. merge rule save করছে
5. নতুন token vocabulary-তে যোগ করছে

১৪) কতগুলো **merge rule** শিখেছে?

```
print(merges)
```

Output:

```

{
('G', 't'): 'Gt',
('i', 's'): 'is',
('e', 'r'): 'er',
('G', 'a'): 'Ga',
('Gt', 'o'): 'Gto',
('e', 'n'): 'en',
('T', 'h'): 'Th',
('Th', 'is'): 'This',
...
}

```

মানে:

এখানে model শিখেছে কোন pair merge করে নতুন token বানাতে হবে।

যেমন:

- ('i', 's') -> 'is'
- ('T', 'h') -> 'Th'
- ('Th', 'is') -> 'This'

এভাবে ধীরে ধীরে বড় token তৈরি হয়েছে।

১৫) Final vocabulary কেমন হলো?

```
print(vocab)
```

Output:

```
[ '<|endoftext|>', ',', '.', 'C', 'F', 'H', 'T', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'k', 'l', 'm', 'n', 'o',  
'p', 'r', 's', 't', 'u', 'v', 'w', 'y', 'z', 'Ġ', 'Ġt', 'is', 'er', 'Ġa', 'Ġto', 'en', 'Th', 'This', 'ou', 'se',  
'Ġtok', 'Ġtoken', 'nd', 'Ġis', 'Ġth', 'Ġthe', 'in', 'Ġab', 'Ġtokeni']
```

মানে:

Final vocabulary গঠিত হয়েছে:

- special token
- initial alphabet
- সব learned merged token

১৬) গুরুত্বপূর্ণ নোট

`train_new_from_iterator()` ব্যবহার করলে exact একই vocabulary নাও আসতে পারে।

কেন?

কারণ যদি একাধিক pair-এর frequency সমান হয়, তাহলে:

- আমাদের code প্রথম যেটা পেয়েছে সেটাই নেয়
- কিন্তু Hugging Face Tokenizers library ভেতরের ID-এর ভিত্তিতে pair বেছে নেয়

অর্থাৎ algorithm একই হলেও implementation detail আলাদা হতে পারে।

১৭) নতুন text tokenize করা

এখন শেখা merge rule ব্যবহার করে নতুন text tokenize করা যায়।

```
def tokenize(text):
    pre_tokenize_result = tokenizer._tokenizer.pre_tokenizer.pre_tokenize_str(text)
    pre_tokenized_text = [word for word, offset in pre_tokenize_result]
    splits = [[l for l in word] for word in pre_tokenized_text]
    for pair, merge in merges.items():
        for idx, split in enumerate(splits):
            i = 0
            while i < len(split) - 1:
                if split[i] == pair[0] and split[i + 1] == pair[1]:
                    split = split[:i] + [merge] + split[i + 2 :]
                else:
                    i += 1
            splits[idx] = split

    return sum(splits, [])
```

এই function কী করে?

নতুন text এলে:

1. আগে pre-tokenize করে
2. প্রতিটি word-কে character-এ split করে
3. শেখা merge rule একে একে apply করে
4. শেষে final token list return করে

১৮) Example: tokenize করা

```
tokenize("This is not a token.")
```

Output:

```
['This', 'Gis', 'G', 'n', 'o', 't', 'Ga', 'Gtoken', '.']
```

ব্যাখ্যা:

এখানে কিছু অংশ merge হয়ে বড় token হয়েছে:

- This
- Ġis
- Ġa
- Ġtoken

কিন্তু not পুরোটা merge হয়নি, তাই সেটা ভেঙে গেছে:

- Ġ
- n
- o
- t

১৯) Unknown character নিয়ে সমস্যা

লেখায় বলা হয়েছে:

আমাদের এই implementation error দেবে যদি unknown character আসে।

কেন?

কারণ আমরা initial vocabulary-তে সব possible character বা byte রাখিনি।

GPT-2-তে এই সমস্যা হয় না, কারণ GPT-2 uses:

byte-level BPE

অর্থাৎ সব possible byte initial vocabulary-তে থাকে, তাই unknown character তৈরি হওয়ার chance থাকে না।

২০) পুরো বিষয়টা খুব সহজ ভাষা

এই implementation-এ BPE algorithm ধাপে ধাপে এভাবে কাজ করে:

Training phase:

- corpus নেওয়া হয়
- pre-tokenize করা হয়
- প্রতিটি word-এর frequency বের করা হয়

- সব unique character নিয়ে base vocabulary বানানো হয়
- word-গুলোকে character-এ split করা হয়
- সবচেয়ে frequent pair খুঁজে merge করা হয়
- নতুন token vocabulary-তে যোগ করা হয়
- এভাবে repeat করা হয়

Tokenization phase:

- নতুন text pre-tokenize করা হয়
- character-এ split করা হয়
- training-এ শেখা merge rule apply করা হয়
- final token তৈরি হয়